SYNAPTIQ DATA PIPELINE EXERCISE · PYTHON 3.12 + PYSPARK 4.2 + DELTA LAKE 4.4

Daily Retail Sales Pipeline & Forensic Data Trap Resolution

Three-Move Functional Medallion Architecture, Parquet Enrichment Lineage, Lazy Catalyst Execution & 14-Trap Audit

Author: Ryan Hammang | Stack: uv · Python 3.12 · PySpark 4.2 · Delta 4.4 | Quality Gate: PEP-257 Clean · 99.64% Test Coverage

34 Raw Rows

100% Row Conservation

24 raw order lines across 2 daily drops + 10 catalog SKUs -> 22 canonical Silver orders + 2 archived CDC rows (0 silently dropped).

3 Moves · 6 Steps

Delta + Parquet Lineage

Move 1 CSV -> Bronze Delta · Move 2 Clean -> 5 Parquet steps + Silver Delta MERGE · Move 3 Join & Agg -> Gold Delta + CSV.

0 Loops · 0 UDFs

Pure Lazy Catalyst

AST-verified DataFrame.transform() chains, higher-order F.filter(), and Window functions. PEP-257 clean & 99.64% coverage.

14 / 14 Traps

17 Reconciled Sales Cells

$1,804.11 net revenue reconciled across 2024-01-01 ($2,559.81, 11 orders) & 2024-01-02 (-$755.70 net after -$1,099 return).

Three-Move Pipeline Structure & Layer Outcomes

99.64% Coverage

Designed for idempotent daily execution, late-arriving restatements, and step-by-step Parquet inspectability

Pipeline Move	Source Input	Materialized Artifacts	Row Count	Engineering Guarantee
Move 1: Bronze Landing	orders_2024-01-01.csv orders_2024-01-02.csv products.csv	warehouse/bronze/orders (Delta) warehouse/bronze/products (Delta)	**24 raw orders** **10 raw SKUs**	PERMISSIVE + _corrupt_record; dynamic drop_date partition overwrite
Move 2: Parquet Steps	bronze/orders bronze/products	intermediate/01_products_cleaned.parquet 01..03_orders_*.parquet (4 steps)	**24 parsed ->** **22 active + 2 CDC**	Lazy checkpoint_parquet combinator; full pre-merge lineage on disk
Move 2: Silver Delta	Reconciled clean orders & standardized products	warehouse/silver/orders (Delta) silver/products & orders_cdc_superseded	**22 unique orders** **10 SKUs · 2 CDC**	ACID DeltaTable.merge() gated on src.drop_date >= tgt.drop_date
Move 3: Gold & Audit	silver/orders LEFT JOIN silver/products	04_joined_order_facts.parquet gold/daily_sales & gold/trap_forensics output/*.csv	**17 Gold cells** **14 Trap rows**	Dynamic impacted order_date refresh (left_semi) + NULLIF AOV guard

Why Local PySpark + Delta Over Cloud Free Edition

- Zero cloud credentials required: reviewers reproduce Day 1 -> Day 2 -> tests via `uv sync && uv run pytest` in seconds.
- Uses real Delta Lake 4.4 (`DeltaTable.merge`, dynamic partition overwrite) and Spark 4.2 ANSI + UTC semantics locally.

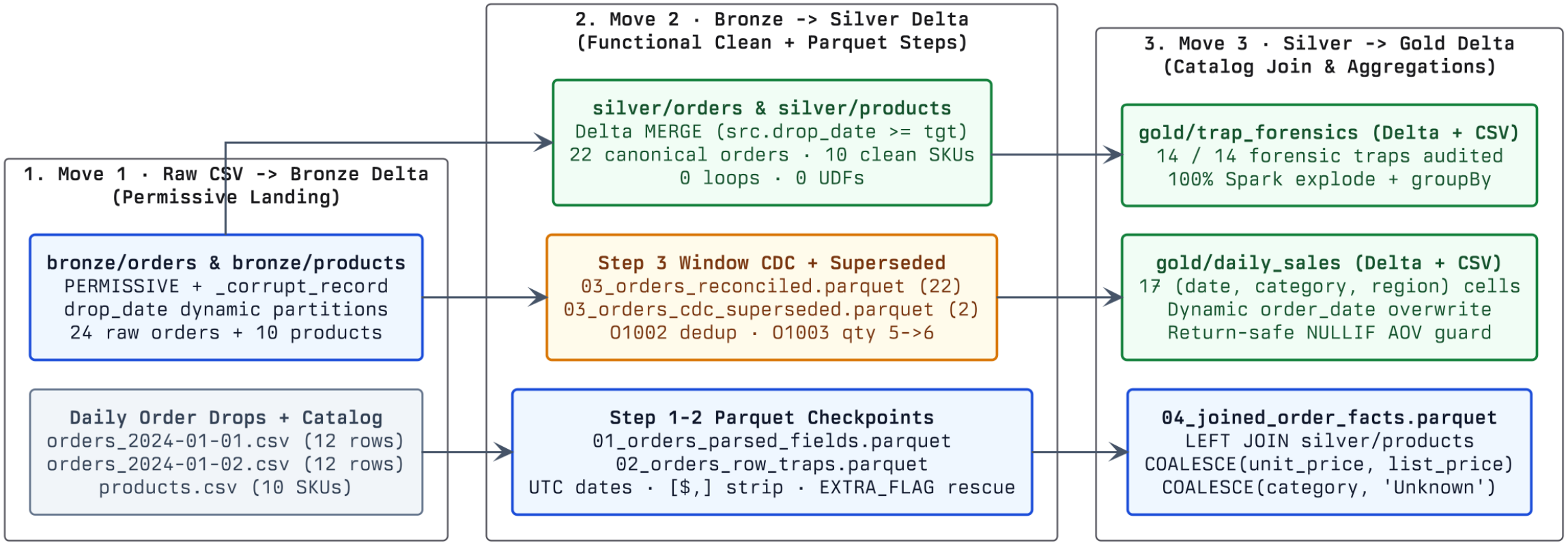
Code Quality & Functional Invariants

- 100% PEP-257 docstring compliance verified by `ruff check --select D` and `pydocstyle --convention=pep257`.
- 99.64% statement coverage (`279 / 280` statements) with AST test blocking `for`, `while`, comprehensions, and UDFs.

End-to-End Medallion & Parquet Enrichment Lineage

6 Parquet Checkpoints

Move 1 (CSV -> Bronze) -> Move 2 (Clean -> Silver Delta) -> Move 3 (Catalog Join & Aggregations -> Gold)



Functional Composition & Lazy Evaluation (0 Loops · 0 UDFs)

AST Enforced in CI

How every transformation avoids Python row iteration and compiles directly to WholeStageCodegen JVM plans

1. Pure DataFrame.transform() Pipelines

Composable DataFrame -> DataFrame Functions

- Bronze, Silver, and Gold steps are stateless functions composed via `df.transform(step_1).transform(step_2)`.
- Catalyst fuses string trimming, regex currency stripping, and multi-format UTC date parsing into single-pass scans.
- Eliminates mutable DataFrame reassignment chains and imperative control flow.

2. Higher-Order Catalyst Functions Over UDFs

Native JVM Array Filtering & Window Specs

- Forensic trap flags use Spark higher-order `F.filter(F.array(...), lambda item: item.isNotNull())`.
- Zero `@udf` or `pandas_udf` boundaries: avoids Python worker pickle serialization and preserves predicate pushdown.
- Cross-file deduplication and restatement detection run via lazy `Window.partitionBy('order_id')`.

3. Lazy Parquet Stage Checkpointing

Inspectable Enrichment Without Re-Scanning CSVs

- `checkpoint_parquet(spark, path)` writes an enrichment step to Parquet and returns a lazy `spark.read.parquet(path)`.
- Downstream branches (Silver MERGE, CDC superseded archive, Gold joins, Trap audit) share materialized Parquet lineage.
- Combines lazy within-stage optimization with deterministic stage-boundary observability.

4. Automated Static & Runtime Guardrails

AST Loop/UDF Gate + PEP-257 + 99.64% Coverage

- `test_zero_loops_and_zero_udfs_in_pipeline_code` walks the Python AST of every module in `src/synaptiq_pipeline/`.
- Rejects `ast.For`, `ast.While`, all comprehension nodes, and any `udf` / `pandas_udf` symbol.
- Spark 4 ANSI mode (`spark.sql.ansi.enabled=true`) + `try_cast` / `try_to_date` prevent silent numeric overflow.

CSV Framing, Currency & UTC Date Traps (#1–#5)

Traps #1 - #5 Fixed

Five ingestion and parsing traps planted in orders_2024-01-01.csv and orders_2024-01-02.csv

#	Forensic Trap	Raw CSV Evidence	Naive Pipeline Failure	Functional PySpark Resolution
1	**Extra Trailing CSV Column**	`O1011 (01-01 L12): ...,24.99,East,EXTRA_FLAG (8 cols)`	FAILFAST crashes job; DROPMALFORMED silently drops valid $24.99 order.	Read PERMISSIVE + _corrupt_record; extract EXTRA_FLAG into rescued_extra via quote-aware regex and keep 7 cols.
2	**Truncated 6-Col Row & Missing Price/Region**	`O2009 (01-02 L12): O2009,2024-01-02,C019,P004,2,`	Crashes strict parsers or drops $59.98 of Home & Office sales via NULL unit_price.	Default missing region -> 'Unknown' in Silver; backfill COALESCE(o.unit_price, p.list_price) ($29.99) in Gold join.
3	**Currency Symbols & Quoted Commas**	`O1001 ($24.99); O1002 & O2004 ("$1,099.00")`	Spark 4 ANSI CAST throws CAST_INVALID_INPUT; naive split(',') shifts columns.	RFC-4180 quote parsing + regexp_replace(trim(unit_price), '[$,]', '') before try_cast('decimal(12,2)').
4	**Ambiguous US Slash Dates**	`O1012 (01/01/2024) & O2001 (01/02/2024 in Jan-2 drop)`	ISO-only to_date returns NULL; dd/MM/yyyy misclassifies Jan 2 order O2001 into Feb 1.	O2001 (O2xxx ID in Jan-2 file) proves MM/dd/yyyy; parse via coalesce(iso, MM/dd/yyyy, epoch_utc).
5	**Unix Epoch Midnight UTC Shift**	`O2003 (01-02 L6): order_date = 1704153600`	Fails date cast, or shifts 2024-01-02 00:00 UTC back to 2024-01-01 in US timezones.	Pin spark.sql.session.timeZone=UTC and convert 10-digit epochs via to_date(timestamp_seconds(bigint)).

Case Drift, CDC, Lagging SKUs & Return Traps (#6–#13)

Traps #6 - #13 Fixed

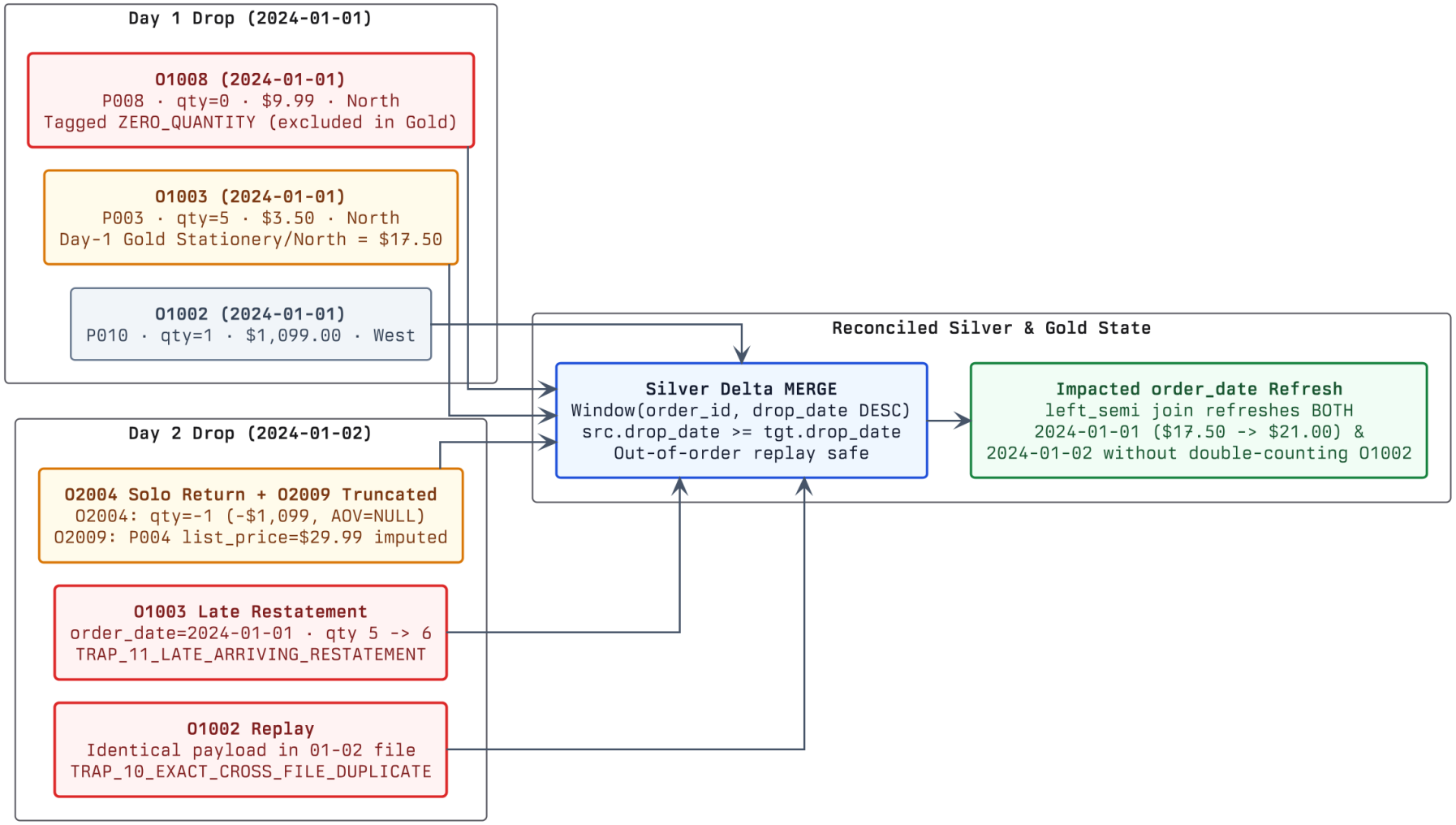
Eight standardization, incremental CDC, referential integrity, and metric-definition traps resolved

#	Forensic Trap	Raw CSV Evidence	Naive Pipeline Failure	Functional PySpark Resolution
6-7	**Region & Category Case Drift**	`west (O1005), east (O2006); electronics (P002), kitchen (P006)`	Splits Gold groups into duplicate lowercase/uppercase rows; naive title() breaks '&'.	Normalize via initcap(lower(trim(...))) in Silver (preserving 'Home & Office' cleanly).
8	**Guest Checkout Blank customer_id**	`O1004 (01-01) & O2005 (01-02): customer_id = ''`	Blanket dropna() / NOT NULL validation discards $80.95 of valid guest sales.	Normalize blank strings to NULL via nullif() and retain guest orders in Silver & Gold.
9	**Zero-Quantity Order AOV Dilution**	`O1008 (01-01): P008, qty = 0, unit_price = 9.99, North`	Adds $0 revenue while inflating Stationery/North order_count (1 -> 2), cutting AOV in half.	Tag order_status='ZERO_QUANTITY' in Silver and filter quantity != 0 in Gold daily_sales.
10-11	**Cross-File Duplicate & Late Restatement**	`01-02 replays O1002 & updates O1003 (01-01) qty 5 -> 6`	Append double-counts $1,099; overwriting only drop_date=01-02 leaves 2024-01-01 stale.	Window dedup + Delta MERGE (src.drop_date >= tgt) + left_semi refresh of impacted order_dates.
12	**Unmapped SKUs Lagging Catalog**	`O1007 -> P099 ($30.00) O2007 -> P011 ($19.99)`	INNER JOIN to products.csv silently loses $49.99 when new SKUs launch before catalog sync.	LEFT JOIN silver/products with COALESCE(category, 'Unknown') so daily revenue reconciles 100%.
13	**Solo Negative Return Bucket AOV**	`O2004 (01-02): qty = -1, -$1,099 (sole West/Elec row)`	Dividing by positive order_count=0 crashes; counting return as +1 order yields -$1,099 AOV.	Include qty<0 in net_revenue & units_sold; count qty>0 in order_count; guard AOV -> NULL when 0.

Incremental Replay & Prior-Day Restatement Flow

Out-of-Order Safe

Verified across Day 1 -> Day 2 sequential runs AND out-of-order Day 1 replay without regression



Four Trade-offs Where Requirements Left Judgment Room

Documented in NOTES.md

Every open business question is explicitly resolved, flagged in metadata, and documented for analyst review

1. O2009 Catalog Price Imputation vs. Quarantine

Why products.csv Includes list_price

- Across all 21 other matched rows, transactional `unit_price` equals `products.list_price` (`0.00` variance).
- Backfilling `O2009` (`P004`) with `list_price = $29.99` and `region = 'Unknown'` preserves `$59.98` visibly.
- Flagged via `is_price_imputed = true`; if discounts existed in prod, we would quarantine missing-price rows instead.

2. Returns in Net Revenue & Solo-Bucket AOV

Why the Prompt Asks for 'Net Revenue'

- No discount/tax columns exist in the schema—`O2004` (`qty = -1`, `-$1,099.00`) is what separates net from gross revenue.
- In `(2024-01-02, Electronics, West)`, `O2004` is the sole row: `order_count` counts positive orders (`0`).
- `F.when(order_count > 0, order_count)` yields `AOV = NULL` instead of crashing on divide-by-zero.

3. LEFT JOIN 'Unknown' Over Silent Revenue Drop

Handling Dimension Lag on P099 & Sequential P011

- `P011` (`O2007` on Day 2) is the next sequential ID after `P010`—classic new-SKU launch lagging nightly catalog drops.
- An `INNER JOIN` would silently under-report company daily revenue by `$49.99` with zero analyst visibility.
- `LEFT JOIN` + `COALESCE(category, 'Unknown')` reconciles 100% to finance while surfacing unmapped SKUs.

4. Order Grain & Multi-Line Future-Proofing

order_id Primary Key Today -> Composite Key Tomorrow

- Every `order_id` in both drops is single-SKU (`O1003` updates `P003` quantity `5 -> 6`), making `order_id` the exact PK.
- Gold already uses `COUNT(DISTINCT CASE WHEN quantity > 0 THEN order_id END)` rather than `COUNT(*)`.
- If multi-item baskets arrive later, only Silver's MERGE key widens to `(order_id, line_id)`.

DELIVERABLE #2 · GOLD ANALYTICS TABLE

Verified Output: daily_sales (17 Cells · $1,804.11 Net)

output/daily_sales.csv

Day 1 (2024-01-01): $2,559.81 across 11 orders (32 units) | Day 2 (2024-01-02): -$755.70 net across 9 orders + 1 return

Date	Category	Region	Net Rev	Ord	Units	AOV
2024-01-01	Accessories	South	**49.95**	1	1	49.95
2024-01-01	Electronics	East	**74.97**	2	3	37.49
2024-01-01	Electronics	West	**2,287.90**	3	12	762.63
2024-01-01	Home & Office	West	**29.99**	1	1	29.99
2024-01-01	Kitchen	East	**36.00**	1	3	36.00
2024-01-01	Kitchen	South	**30.00**	1	4	30.00
2024-01-01	Stationery	North	**21.00**	1	6	21.00
2024-01-01	Unknown	East	**30.00**	1	2	30.00

Date	Category	Region	Net Rev	Ord	Units	AOV
2024-01-02	Accessories	West	**49.95**	1	1	49.95
2024-01-02	Electronics	East	**74.97**	1	3	74.97
2024-01-02	Electronics	North	**44.95**	1	5	44.95
2024-01-02	Electronics	West	**-1,099.00**	0	-1	NULL
2024-01-02	Home & Office	Unknown	**59.98**	1	2	59.98
2024-01-02	Kitchen	South	**24.00**	1	2	24.00
2024-01-02	Kitchen	West	**22.50**	1	3	22.50
2024-01-02	Stationery	East	**46.96**	2	6	23.48
2024-01-02	Unknown	South	**19.99**	1	1	19.99

Highlighted left: 2024-01-01 Stationery/North updated $17.50 -> $21.00 on Day 2 (O1003 restatement).
Highlighted right: O2004 solo return & O2009 imputed price.

Scaling to Databricks Unity Catalog (100M+ Rows/Day)

Production Ready

How each local component maps 1:1 onto managed Databricks Lakeflow, Auto Loader, and Unity Catalog primitives

1. Auto Loader (cloudFiles) + _rescued_data

Replaces CLI CSV File Arguments

- Ingest continuously from S3/ADLS Unity Catalog Volumes with `cloudFiles.schemaEvolutionMode = 'rescue'`.
- RocksDB checkpointing tracks millions of landed drops with exactly-once guarantees—no full Bronze scans.
- Trailing fields like `O1011`'s `EXTRA_FLAG` land automatically in `_rescued_data` JSON.

2. Lakeflow AUTO CDC & SCD Type 2 Catalog

Incremental Streaming MERGE & Historical Price/Category Fidelity

- Stream Bronze -> Silver via Lakeflow `APPLY CHANGES INTO ... SEQUENCE BY (drop_date, ingested_at)`.
- Model `silver.products` as SCD Type 2 (`effective_from`, `effective_to`) so future category/price edits don't rewrite past quarters.
- Automatically re-enrich `'Unknown'` SKU rows (`P099`, `P011`) when delayed catalog files arrive.

3. Unity Catalog, Liquid Clustering & Expectations

Storage Layout & Automated Data Quality SLAs

- Replace Hive `partitionBy('order_date')` with Delta Liquid Clustering `CLUSTER BY (order_date, category, region)`.
- Eliminates small-file amplification while accelerating date, category, and region BI slicing.
- Enforce Delta `CHECK` constraints and Lakeflow Expectations with Slack/PagerDuty alerts on quarantine spikes.

4. Databricks Asset Bundles & File Arrival Triggers

CI/CD Deployment & Event-Driven Scheduling

- Package code, tests, and job definitions declaratively in `databricks.yml` across `dev` / `staging` / `prod`.
- Use Workflows **File Arrival Triggers** on the landing Volume instead of fixed cron schedules.
- Publish `gold.daily_sales` as a Serverless Materialized View / AI-BI Dashboard with zero manual exports.

Test Coverage (99.64%), PEP-257 & AST Purity Gate

279 / 280 Stmts (99.6%)

Measured via `uv run pytest` (--cov-fail-under=80), `ruff check --select D`, and `pydocstyle --convention=pep257`

Module	Stmts	Miss	Cover	Missing Lines / Note
src/synaptiq_pipeline/__init__.py	4	0	100%	(none)
src/synaptiq_pipeline/bronze.py	25	0	100%	(none)
src/synaptiq_pipeline/cli.py	34	1	97%	Line 99: if __name__ == '__main__':
src/synaptiq_pipeline/gold.py	38	0	100%	(none)
src/synaptiq_pipeline/paths.py	55	0	100%	(none)
src/synaptiq_pipeline/pipeline.py	26	0	100%	(none)
src/synaptiq_pipeline/silver.py	62	0	100%	(none)
src/synaptiq_pipeline/spark.py	36	0	100%	(none)
TOTAL (Gate >= 80.0%)	**280**	**1**	**99.64%**	**+19.64 pts above 80% requirement**

PEP-257 Docstring Gate

0 Violations Across src/ & tests/

- Enforced via `[tool.ruff.lint.pydocstyle] convention = 'pep257'` (`D` rules) and `pydocstyle`.
- Every module, class, method, property, and function carries imperative PEP-257 docstrings.

AST Zero-Loop / Zero-UDF Gate

Static Python AST Enforcement

- Parses `src/synaptiq_pipeline/*.py` via `ast.parse`.
- Fails if any `For`, `AsyncFor`, `While`, comprehension, or `udf` / `pandas_udf` node appears.

4 End-to-End PySpark Suites

Real JVM Spark 4.2 + Delta 4.4

- Tests Day 1 -> Day 2 restatement (`O1003` 5->6), out-of-order replay, all 14 traps, `O2004` NULL AOV, and CLI.

Runtime Deployment & Functional Component Topology

PlantUML allowmixing

Combines UML deployment nodes, artifacts, folders, and Delta databases with Python 3.12 / JVM Spark 4.2 components

